1

```
from pyspark.sql import SparkSession
from collections import defaultdict
import math

# Initialize Spark
spark = SparkSession.builder.appName("StackOverflowAnalysis").getOrCreate()
sc = spark.sparkContext
```

2

```
# Load CSV as RDD
path = "/FileStore/tables/shortStackOverflow.txt"  # Change path as needed
lines = sc.textFile(path)
```

3

```
# Step 1: Parse the CSV lines into usable data structures
def parse_line(line):
    parts = line.strip().split(",")
    post_type = int(parts[0])
    post_id = parts[1]
    accepted_answer = parts[2] if parts[2] != '' else None
    parent_id = parts[3] if parts[3] != '' else None
    score = int(parts[4]) if parts[4] else 0
    tag = parts[5] if len(parts) > 5 else None
    return (post_type, post_id, accepted_answer, parent_id, score, tag)

posts = lines.map(parse_line)
```

4

```
# Step 2: Map posts to (question_id, post_data)
def mapper(post):
    post_type, post_id, accepted_answer, parent_id, score, tag = post
    if post_type == 1 and tag:  # Question
        is_solved = accepted_answer is not None
        return [(post_id, ("question", 0, is_solved, tag))]
    elif post_type == 2:  # Answer
        return [(parent_id, ("answer", 1))]
    return []

mapped = posts.flatMap(mapper)
```

;

```
# Step 3: Reduce by question_id to group answers with their question
def reducer(a, b):
    if a[0] == "question":
        tag = a[3]
        is_solved = a[2]
        return ("question", a[1] + (b[1] if b[0] == "answer" else 0), is_solved, tag)
    elif b[0] == "question":
        tag = b[3]
        is_solved = b[2]
        return ("question", b[1] + (a[1] if a[0] == "answer" else 0), is_solved, tag)
    else:
        return ("answer", a[1] + b[1])

reduced = mapped.reduceByKey(reducer)
```

```
# Step 4: Map to (tag, (solved_count, solved_total_answers, unsolved_count,
unsolved_total_answers))
def tag_mapper(data):
    _, (typ, answer_count, is_solved, tag) = data
    if typ != "question": return []
    if is_solved:
        return [(tag, ("solved", 1, answer_count))]
    else:
        return [(tag, ("unsolved", 1, answer_count))]

tagged = reduced.flatMap(tag_mapper)
```

```
    # Step 5: Aggregate by tag
    def combiner(a, b):
        if a[0] == "solved" and b[0] == "solved":
            return ("solved", a[1] + b[1], a[2] + b[2])
        elif a[0] == "unsolved" and b[0] == "unsolved":
            return ("unsolved", a[1] + b[1], a[2] + b[2])
        elif a[0] == "solved" and b[0] == "unsolved":
            return ("mixed", (a[1], a[2]), (b[1], b[2]))
        elif a[0] == "unsolved" and b[0] == "solved":
            return ("mixed", (b[1], b[2]), (a[1], a[2]))
        elif a[0] == "mixed":
            if b[0] == "solved":
                return ("mixed", (a[1][0] + b[1], a[1][1] + b[2]), a[2])
            else:
                return ("mixed", a[1], (a[2][0] + b[1], a[2][1] + b[2]))
        elif b[0] == "mixed":
            return combiner(b, a)

    aggregated = tagged.reduceByKey(combiner)
```

```
    # Step 6: Format results
    def final_format(tag_data):
        tag, stats = tag_data
        if stats[0] == "solved":
            solved_count, solved_total = stats[1], stats[2]
            unsolved_count, unsolved_total = 0, 0
        elif stats[0] == "unsolved":
            solved_count, solved_total = 0, 0
            unsolved_count, unsolved_total = stats[1], stats[2]
        else:
            solved_count, solved_total = stats[1]
            unsolved_count, unsolved_total = stats[2]

        avg_solved = round(solved_total / solved_count, 1) if solved_count > 0 else float('nan')
        avg_unsolved = round(unsolved_total / unsolved_count, 1) if unsolved_count > 0 else
    float('nan')

        return (tag, (solved_count, avg_solved, unsolved_count, avg_unsolved))

    results = aggregated.map(final_format)
```

```
Java 3 2.0 | 1 0.0
PHP 2 1.5 | 0 nan
Python 1 3.0 | 1 2.0
C# 0 nan | 2 3.5
```